PART 5 OF 5. GUARDRAILS, FAILURE, AND WHAT TO BUILD SECOND.

The rules I do not break

Everything up to here was about making the thing work. This part is about making sure it never does damage, because a tool that touches other people's work and other people's names can do real harm quietly.

Every rule below cost me something to learn. Two of them I got wrong first and had to go back and undo. None of them are technical.

Then the second build, which is the one that gets you the rest of your week back.

1 Never automate a judgment

A program can tell you an item is late. It cannot tell you whether that matters, why it slipped, or what to do about it.

Late because somebody is stuck on a dependency, late because the estimate was always wrong, and late because nobody has looked at it in a month are three different situations that look identical in a query. The program surfaces the fact. You supply the meaning.

I wanted an exception to this. When somebody replies, I wanted the item to move from not started to in progress automatically, because otherwise the status column is dead weight and a moving item looks untouched. Then I wrote out the replies I actually get. Haven't started, picking it up Monday. This isn't mine. Blocked on somebody else. No update this week. Every one of those would have been marked in progress, and every one of them means the opposite.

> EXAMPLE
> What I do instead is move it exactly one step, to acknowledged. A reply proves somebody has seen the item. That is all it proves, and it is worth something on its own.

WHY THIS MATTERS: in the tracker a wrong state is survivable, because the person's own words sit next to it and contradict it. On a slide in front of leadership, the words are not in that column. The state goes up on its own.

2 Delete your escape hatch, do not just avoid using it

My first version let me resolve a reply the system could not match by supplying the item ID myself. Convenience. I used it once during testing and it made a refused write look like a successful one, which hid the fact that the guard had worked exactly as designed.

So I deleted it. Not disabled, not commented out, not hidden behind a setting. Removed, so that passing those arguments is now an error.

> EXAMPLE
> If somebody sends me a plain message asking what this is, I do not want that to become their status. There is no path for it to. The system either has an exact match or it has nothing.

WHY THIS MATTERS: a safety rule that depends on you being disciplined at 6pm on a Friday is not a safety rule. I chose a system that is never wrong and sometimes empty over one that is usually right.

3 A control that does nothing is worse than no control

I had a setting in my config file that was supposed to hold writes back for review. It was in the config. It printed in the status output. It was referenced in five different documents.

No code read it. It had never done anything. Every write it was supposed to be gating had gone through the entire time, and I had been reading its name in the status output and feeling reassured.

> EXAMPLE
> I deleted the setting rather than wiring it up, and changed the status output to describe what the code actually does instead of what the config claimed. Then I went and checked every other switch the same way.

WHY THIS MATTERS: the danger was never the writes going through. It was that I believed a protection existed, so I stopped watching the thing it was supposed to protect. Go and test that your guardrails fire. Break something on purpose and confirm it gets caught.

4 Escalate on silence, never on lateness

When you send a second reminder, it is tempting to make it louder because the item is further past its date. That is the wrong trigger.

Count unanswered asks instead, measured against the last time anybody actually responded. Somebody who replied yesterday gets a normal message even if their item is badly overdue, because they are engaging and there is nothing to escalate. Somebody who has ignored three asks gets the marked one, even if their date is weeks away.

> EXAMPLE
> My counter resets to one the moment anybody replies. The marker states only what I can prove, which is how many asks went unanswered and the date of the first one. It never says the work is late, blocked or at risk, because I do not know that.

WHY THIS MATTERS: escalating on lateness shouts at the people who are already responding, and they learn to ignore you. Urgency that is always on is not urgency, it is noise with a red icon on it.

5 Build the dashboard second, and never first

Once the reminders work, the same query gives you a status view. What is in each stage, what is late, what is blocked, and the direction of travel over the last few weeks.

Everybody wants to build this first because it is the visible, impressive part. Built first it is a page nobody opens, fed by data nobody maintains. Built second, on data the reminders have been cleaning up for a month, it becomes the thing that replaces asking for status out loud.

> EXAMPLE
> The reminders deleted my chasing. The dashboard deleted the part of every meeting where everybody reads their status into the room. Those are two different hours in my week and they came in that order for a reason.

WHY THIS MATTERS: a dashboard is only as good as the freshness underneath it, and the reminders are what create the freshness. In the other order you get a beautiful page full of stale numbers, which is worse than no page, because people believe it.

One more thing, and it is the one I was most wrong about.

When I proposed this, I argued that my program was visible enough that nobody would push back on getting automated messages. I was told that was exactly backwards. High visibility raises the cost of a wrong record and makes pushback quieter, not absent. People do not complain about a tool like this. They just stop replying, and you find out months later.

So I started small, with people who agreed to it, on work that did not matter much. That was the right call and it was not my instinct.

I built the second one for a different program on the same bones. A rule, three fields, a query, a private message, a page. It took a fraction of the time because the thinking was already done, and the thinking was always the whole thing.

The people who lose to automation are not the ones whose work got automated. They are the ones who never tried to automate anything.